Group-7

CallHub Phone Directory Management System - Module B

Github link: https://github.com/arpitaa-k/CallHub-CS-432.git

Video Link: db_data_dynamics_A2_ModuleB.mp4

Table of contents page:

Table of contents page: 1
Introduction 3
Methodology 3
1. System Architecture 3
2. Local Environment + Data Management 4
   a. Setup: 4
   b. Member Creation/Deletion: 4
   c. Data Integrity 4
Functionality 4
1. API + UI Development 4
   ○ APIs Implemented 5
   ○ UI Panels 7
   ○ Session Validation 7
   ○ Portfolio 7
2. Core Functionalities Delivered 7
   ○ Search & Filtering 7
   ○ RBAC for Sensitive Data 8
   ○ Usage Analytics 8
   ○ Emergency Contact Retrieval 9
   ○ Profile Management (CRUD) 9
Approach 9
1. RBAC & Security 9
   a. Role Semantics 9
   b. Granularity 10
   c. Audit Logging 10
2. API + Security Logs 10
   a. Verification 10
   b. Unauthorized Modification Detection Model 11

Overview of our CallHub Phone Directory Management System **11**
Login 11
Results **19**
1. API Endpoint Latency 19
Observations 19
Reasoning 20
2. SQL Execution Time Analysis 22
Observations 22
Interpretation 22
3. Bottleneck Identification 22
4. Indexing Strategy 23
5. Query Execution Plan Analysis (EXPLAIN) 24
Conclusions **25**
Demonstration Video **25**
Group Contribution: **25**

Introduction

In this assignment, we implement a complete Local API along with a Web UI for the CallHub directory system. The system ensures strict Role-Based Access Control (RBAC), strong data integrity, and comprehensive auditing mechanisms.

It maps to the assignment structure in subtasks and includes Task 1 database schema from Module A.

Methodology

For this assignment, we implemented the system using Flask, MySQL, and a web-based UI, along with a structured database design and data management techniques.

1. System Architecture

a. **Backend:** Python Flask + MySQL
b. **Frontend:** HTML + JavaScript + CSS (Vanilla)
c. **Database Tables:**
   Members, Departments, Roles, Role_Permissions
   Member_Role_Assignments, Contact_Details, Locations, Emergency_Contacts
   User_Credentials, Search_Logs, Audit_Trail
d. **Authentication:**
   Session-based authentication using `login_required` decorator
e. **RBAC Implementation**
   - Fetched **can_edit_others()** from Roles table to give edit access
   - Fetched **can_view()** from Role_Permissions table to check Role permission mapped to 5 categories

- **Audit:**
   - Database audit: **Audit_Trail** table

- File audit: **audit.log** in backend folder

2. Local Environment + Data Management

a. **Setup:**

i. Local MySQL DB created and seeded scalably

ii. Core tables and project-specific details implemented exactly as required

b. Member Creation/Deletion:

i. **Create Member:**

1. **POST /members** (Create): checks and inserts into member + contact + location + emergency + credentials + role assignment

2. Ensured no duplicates in **User_Credentials** for member data

ii. **Delete Member (Soft Delete):**

1. **DELETE /members/<id>** (Soft delete): **sets is_deleted=1**, **deleted_at=NOW()**, audit logged

c. Data Integrity

i. Foreign keys ensure referential integrity

ii. Cascading applied for dependent tables

iii. Soft delete preserves historical data for auditing

Functionality

The developed system supports various functionalities such as member management, search operations, portfolio viewing, and role-based access control.

1. API + UI Development

- **APIs Implemented**
  - **User Authentication & Session API (auth_routes.py)**

    These endpoints manage the security layer and user sessions

    - **POST /login:** Validates credentials. Sets user_id, member_id, and role in the Flask session.
    - **POST /register**: Creates new credentials in the User_Credentials table with a hashed password.
    - **GET /logout:** Clears the session and redirects to the landing page.
    - **POST /keepalive:** Refreshes the last_active timestamp to prevent automatic session timeout (30-minute window).
    - **GET /check-admin:** Returns a JSON object confirming if the logged-in user has administrative (can_edit_others) privileges.
  - **Member Management & CRUD API (member_routes.py)**

    These endpoints handle the data for the directory. "Write" actions are protected by the can_edit_others RBAC check.

    - **GET /search**: Searches members by name prefix. It calculates "frequency" based on Search_Logs to rank popular results.
    - **GET /members**: Fetches a list of members. Admins see all; regular users see only their own basic info.
    - **POST /members:** (Admin Only) Creates a new member record across multiple tables (Contact, Location, Emergency) in a single atomic transaction.
    - **GET /members/<int:id>:** Fetches the full core profile for a specific member ID.

- P**UT /members/<int:id>:** (Admin Only) Updates all data fields for an existing member.
- **DELETE /members/<int:id>:** (Admin Only) Performs a Soft Delete by setting is_deleted = 1 and logging the event in the Audit_Trail.
- **GET /editable-roles:** Returns a list of all role titles/IDs available for the creation/update forms.

- **Privacy-Controlled Portfolio API (portfolio_routes.py)**

  This is the core "Privacy Engine" of the application.

  - **GET /member/<int:member_id>/portfolio**:Retrieves deep profile data (Contacts, Locations, Emergency Contacts).
  - **Logic:** It queries the Role_Permissions for the caller and only returns data categories (e.g., "Official Email" vs "Home Phone") that the caller's role is permitted to see.

- **Web UI / Page Routes (main.py)**

  These serve the HTML templates to the browser. All routes except / require an active session.

  - **GET /:** The landing/login page.
  - **GET /home:** The main dashboard (Homepage.html) showing the user's name and role.
  - **GET /read:** The directory search and view interface.
  - **GET /create: (Admin Only) :**The form to add a new member.
  - **GET /update: (Admin Only):** The form to edit existing member details.
  - **GET /delete: (Admin Only):** The management interface for removing members.

- **UI Panels**
  - **Homepage:** Login and navigation
  - **Read Panel:** Search based on frequency of user searched, role filter, portfolio view
  - **Create Panel**: Role selection and form submission
  - **Update/Delete Panels:** Role-based operations
  - **Portfolio Page:** Displays member details with RBAC filtering

- **Session Validation**
  - **utils/auth.py** provides **login_required** check via Flask session
  - Each route uses it to authenticate
  - API requests via JS use **credentials: include**

- **Portfolio**

  Portfolio will return:

  - Member core fields
  - Contacts / locations / emergency_contacts filtered by role categories
  - Department info
  - UI hides empty category sections

2. Core Functionalities Delivered

The system provides a comprehensive set of functionalities to efficiently manage and access member information within the directory.

- **Search & Filtering**

  Functionality: The system allows users to search member details based on the frequency of user searches using name-based queries and supports role-based filtering of results.

  Features:

  - Name prefix search

- Role-based filtering
- Frequency based search
- Extendable to department

Approach: This is implemented using a centralized **`/search`** API that performs SQL queries on the Members table. RBAC constraints are applied by joining with role-related tables to ensure users only see authorized data.

Code References:

- **`routes/member_routes.py`**
- **`templates/read.html`**
- **`utils/logger.py`** ‘

RBAC for Sensitive Data

Functionality: Controls access to data based on user roles such as Admin and Regular users.

Approach: RBAC is implemented using the **`Roles`** and **`Role_Permissions`** tables. Permissions are checked in backend logic using helper functions **`can_edit_others()`**and **`can_view()`**, and conditions are enforced in APIs to restrict unauthorized actions.

Code References:

- **`utils/rbac.py`**
- **`routes/portfolio_routes.py`**
- **`templates/read.html`**

Usage Analytics

Functionality: Tracks user actions such as searches and system usage patterns.

Approach: Implemented using the `Search_Logs` table, where each search query is recorded. Aggregation queries can be used to analyze usage trends and frequently searched data

Code References:

- `Search_Logs` table
- `Audit_Trail`

- `audit.log`

Example Queries:

SELECT searched_term, COUNT(*) FROM Search_Logs GROUP BY searched_term;

SELECT category_id, COUNT(*) FROM Audit_Trail WHERE action='SOFT_DELETE' GROUP BY category_id;

Emergency Contact Retrieval

Functionality: Provides quick access to emergency contact details of members

Approach: Implemented using optimized SQL joins between Members and Emergency_Contacts tables. Indexing on foreign keys ensures faster query execution.

Profile Management (CRUD)

Functionality: Displays complete member information including contacts, locations, emergency details, and department data.

Approach: Implemented using the `/member/<id>/portfolio` API, which performs multiple joins across tables. RBAC filters are applied to restrict sensitive categories, and the frontend dynamically hides empty sections.

- `POST, PUT, DELETE` endpoints
- Admin-only modification
- Soft delete with audit preservation

Approach

1. RBAC & Security

While developing the system, special focus was given to security, access control, and proper handling of user data

a. Role Semantics

- `Roles` table includes `can_edit_others`

- `Role_Permissions` maps category access (public/academic/etc) includes can_view()

b. Granularity

1. Director, Dean Academics, Dean Student Welfare, Registrar, HOD are defined as admin in the app for editing
2. Regular user can only view own details via `/members/<id>` with `actor_id == id` condition

c. Audit Logging

1. `utils/logger.py have function log_action()` logs DB and file audit
2. `Audit.log` contains:
   a. timestamp : Records the exact date and time when an action occurred.
   b. actor : Identifies the user who performed the action.
   c. table : Specifies which database table was modified.
   d. action : Describes the operation executed (INSERT, UPDATE, DELETE, SOFT_DELETE, SELECT, LOGIN, LOGOUT, SEARCH)
   e. Source (API/Manual): Indicates whether the operation was executed via API or directly on the database.
3. `Audit_Trail` table includes `source` (API vs manual database audit)

2. API + Security Logs

a. Verification

1. Protected endpoints return 403 for locked accounts
2. Unauthorized actions return JSON errors
3. Logs available in:
   a. callhub_backend/audit.log
   b. Audit_Trail table

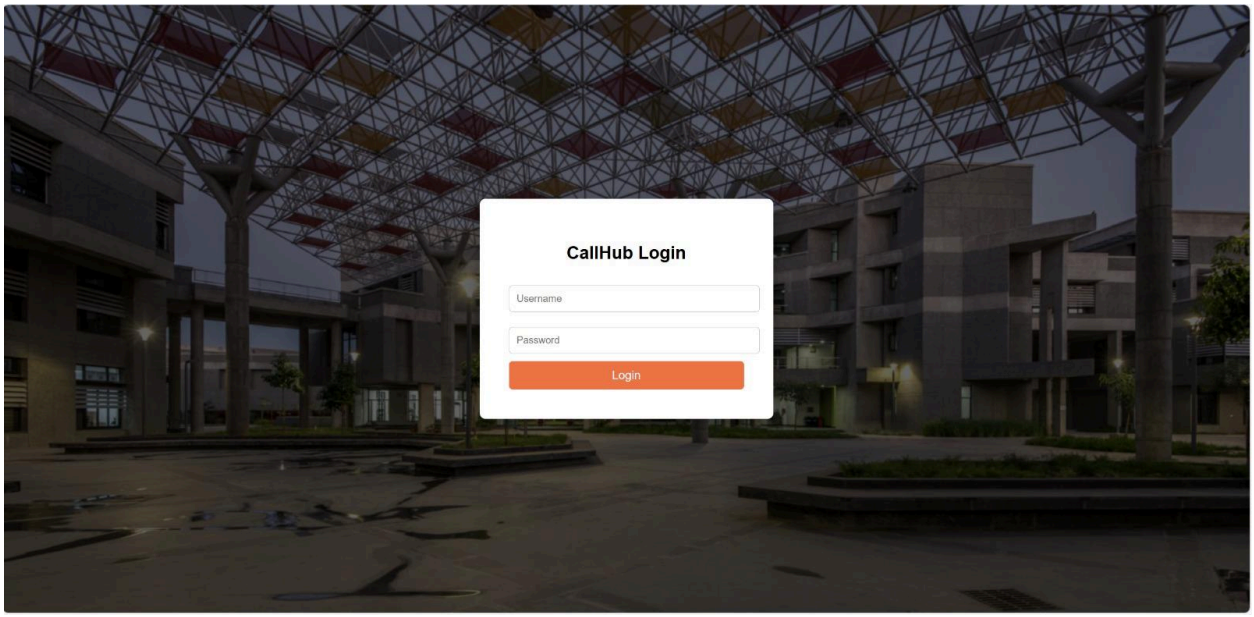
b. Unauthorized Modification Detection Model

```
1. source = 'api' for API-based audit entries
2. Direct DB writes → source = DB
3. Enables detection of unauthorized DB modifications
```

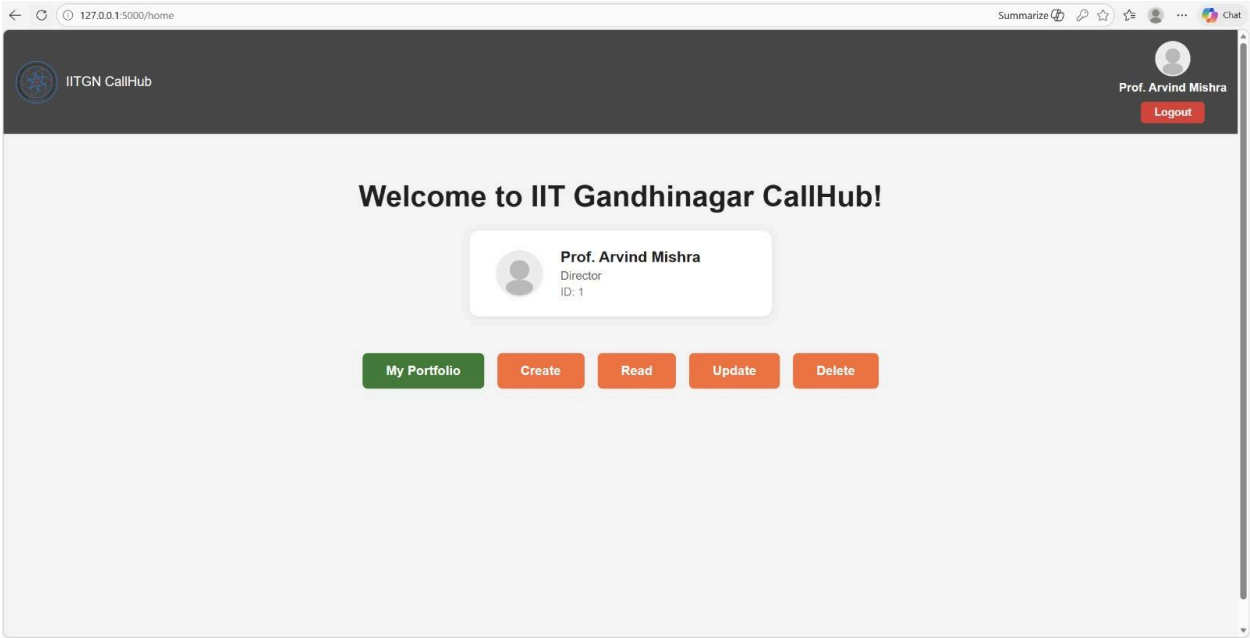
Overview of our CallHub Phone Directory Management System

Login

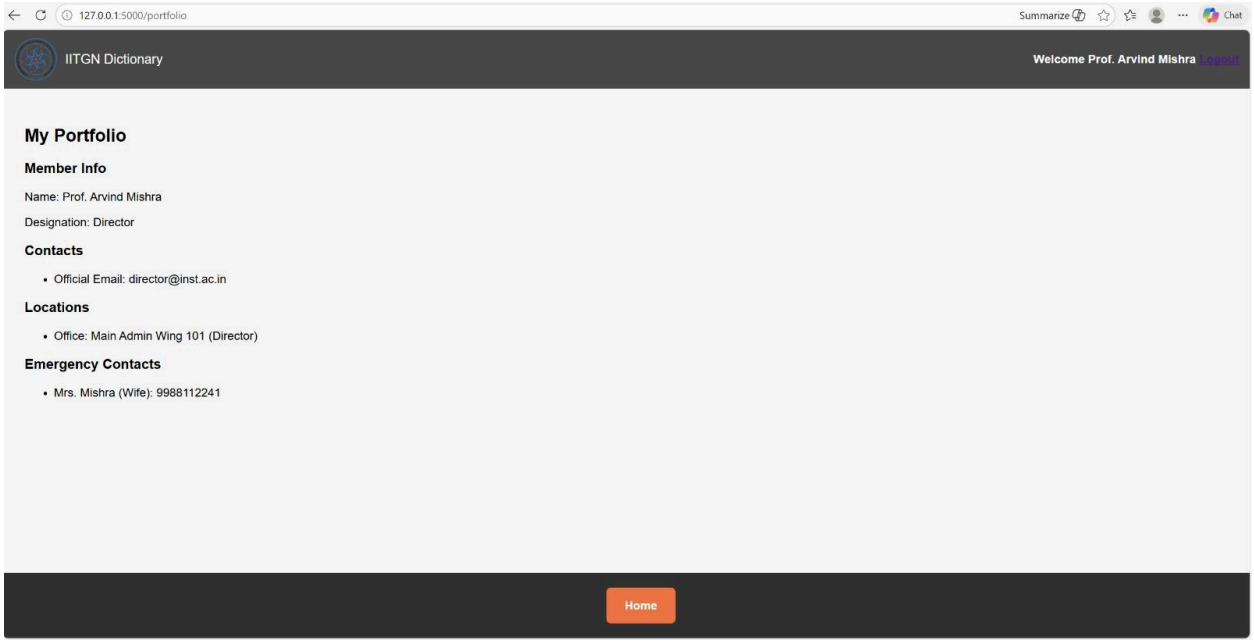
- Front login page with username and password



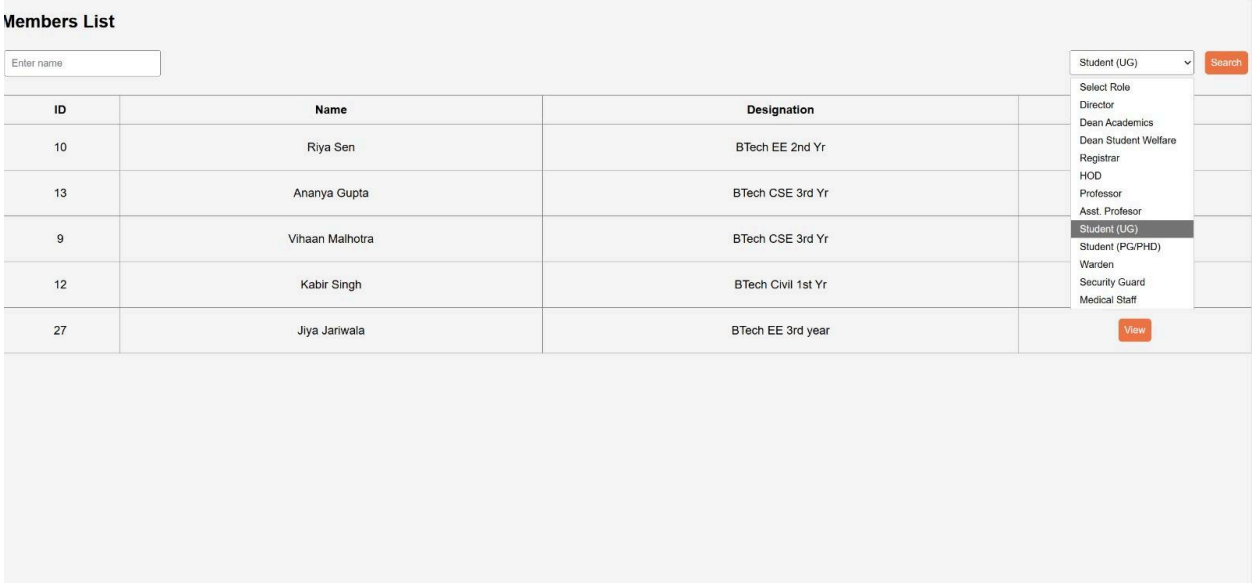
- After logging in, we can see logged in user's info in **My Portfolio.**



- My Portfolio page contains all logged-in user details that the logged-in user has can_view() access to.

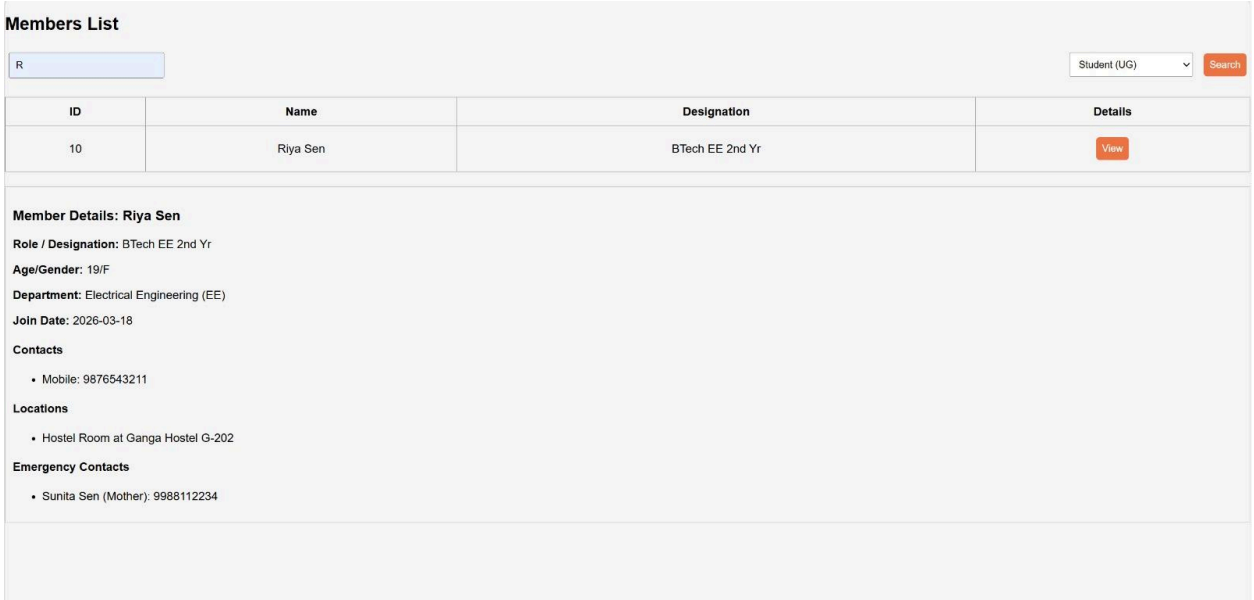


- After going into the Read page, this page will be shown. Here, role-based search is possible.



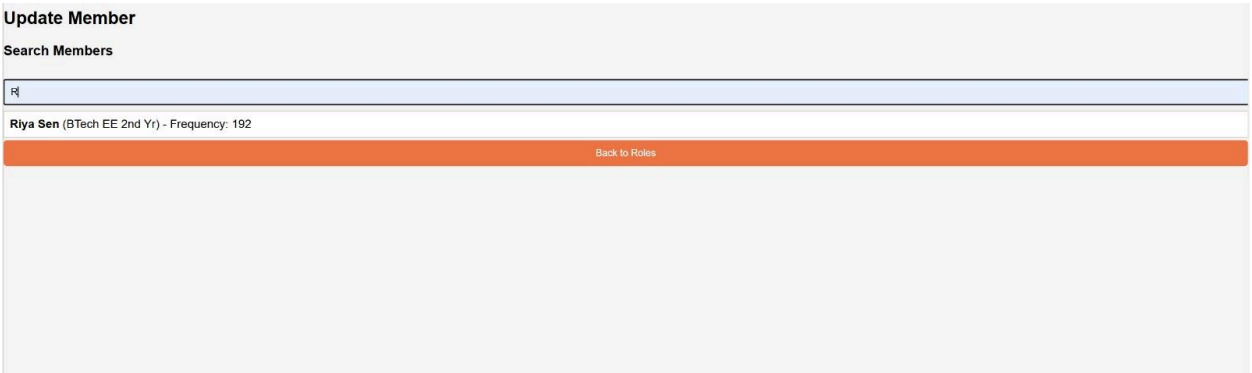
- Can search in the Enter name tab, which will also show frequently searched users in order.

- When clicked on View, the portfolio of the searched user will be displayed based on the logged-in user's view access.

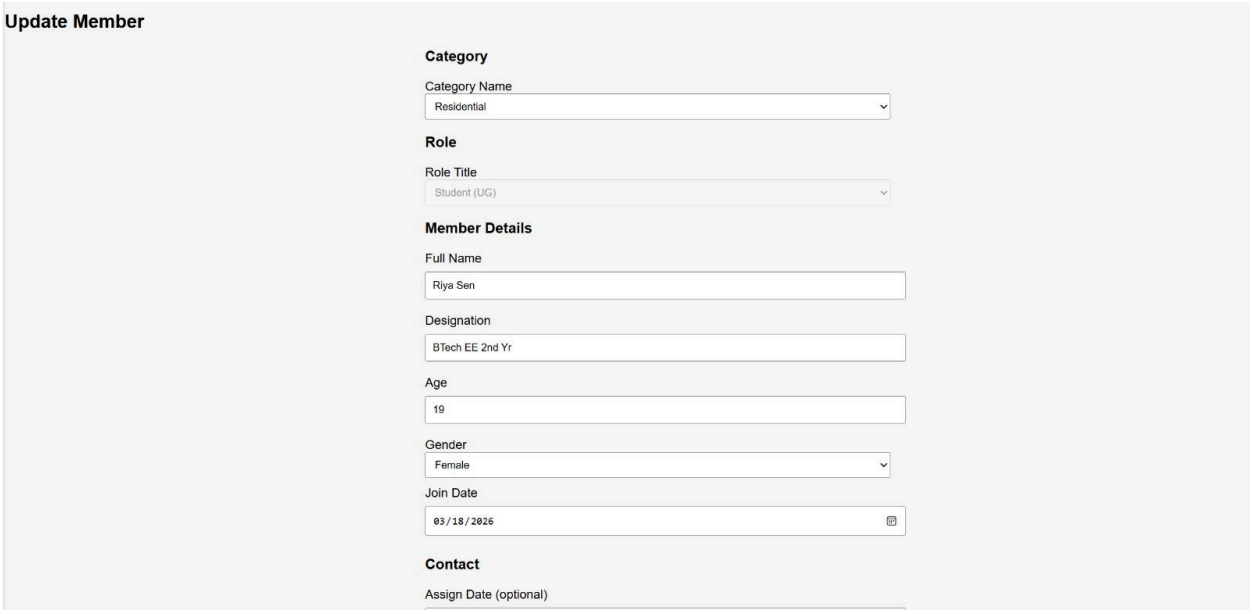


- This page will be shown only after the logged-in user clicks the Update button, if the logged-in user has edit access. Here we can select the role to update.

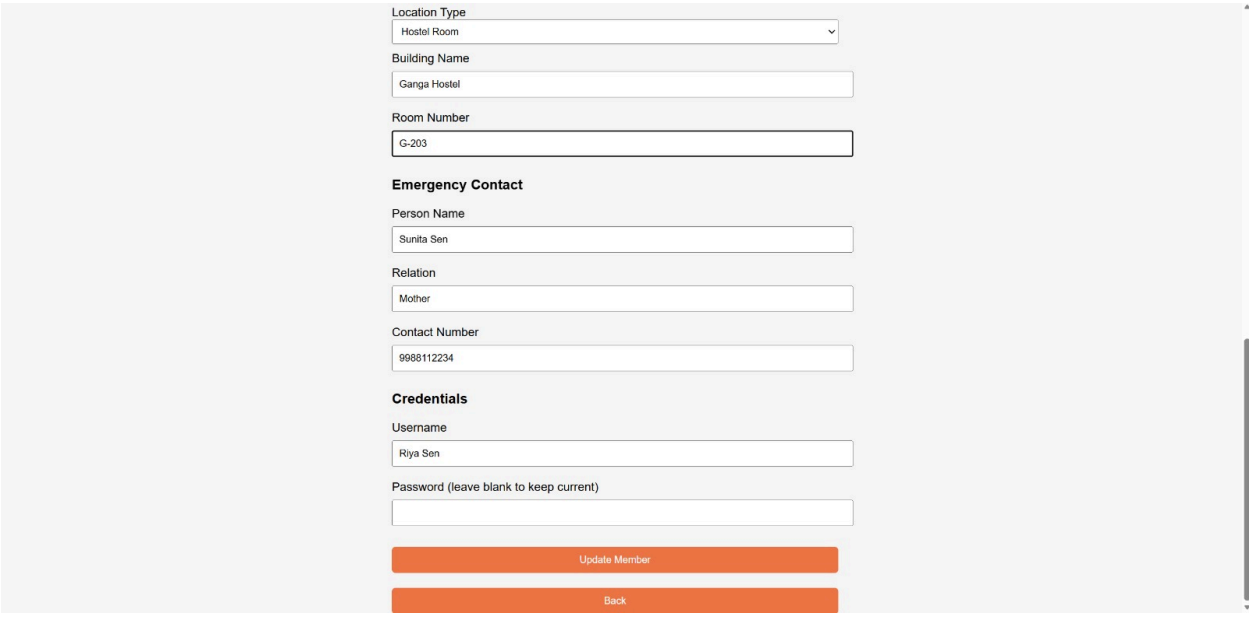
- After selecting a role, members of that role can be selected to update. Here also the frequency of that searched user will be shown.



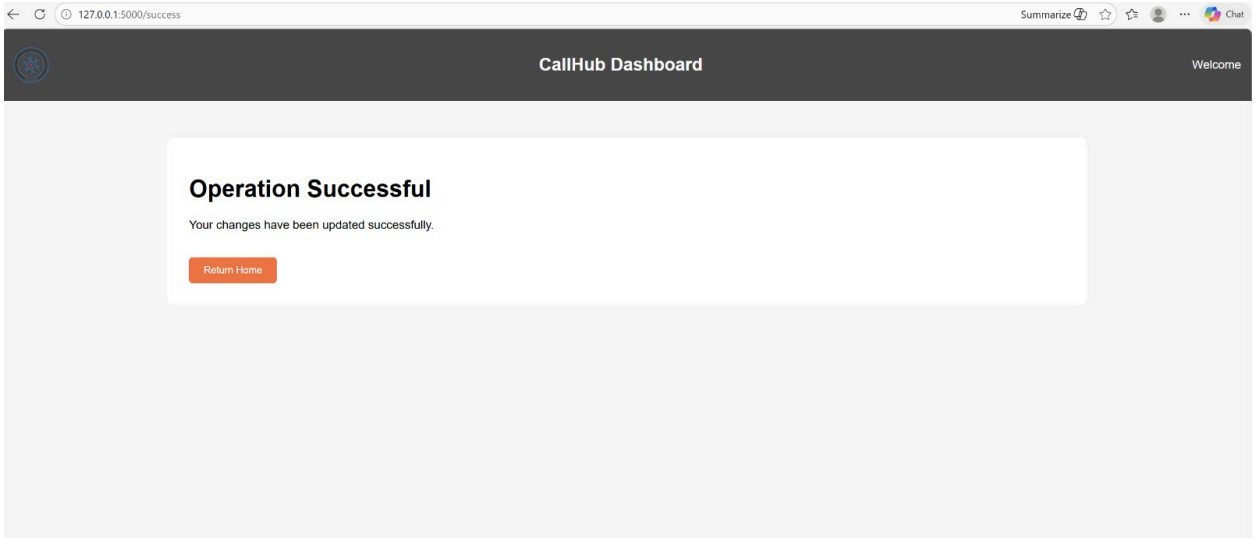
- This is the page after selecting the member to update. Here the user can modify any data of the selected member according to what access he/she has.



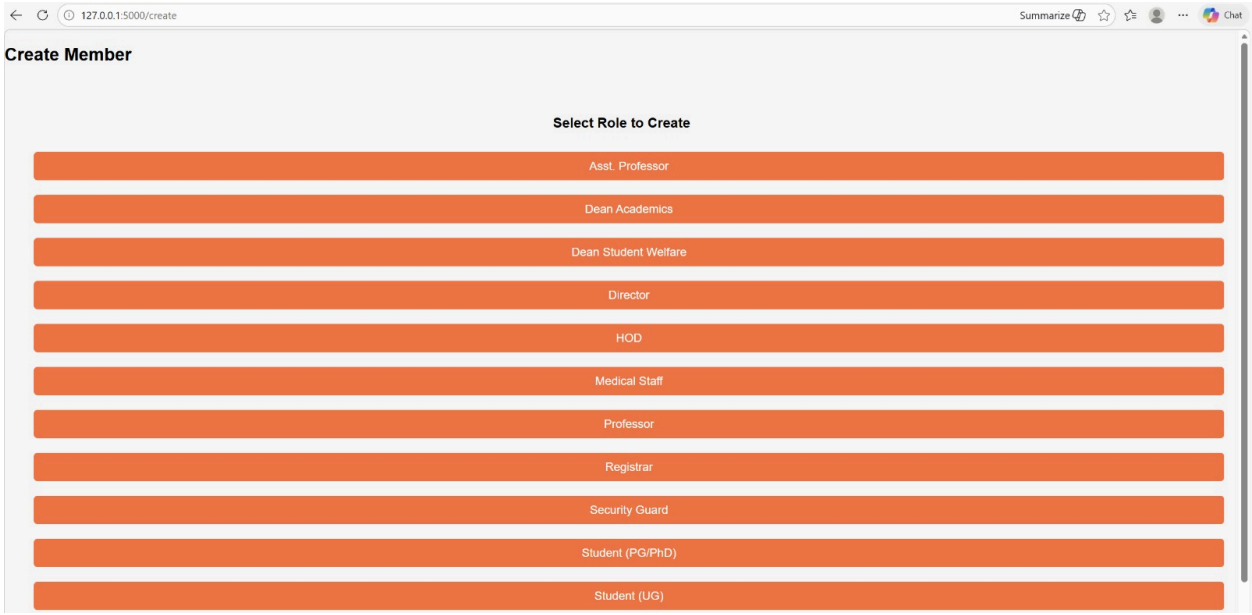
- After updating the data , click on the update member button.



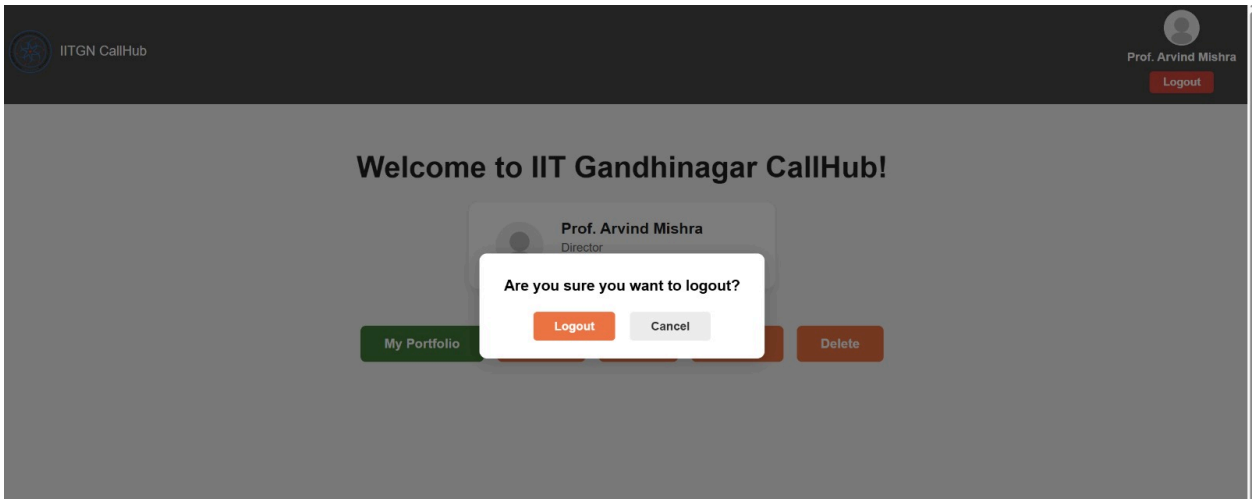
- This will take us to the Operation Successful page. Similarly , after deleting a member this page will be shown.



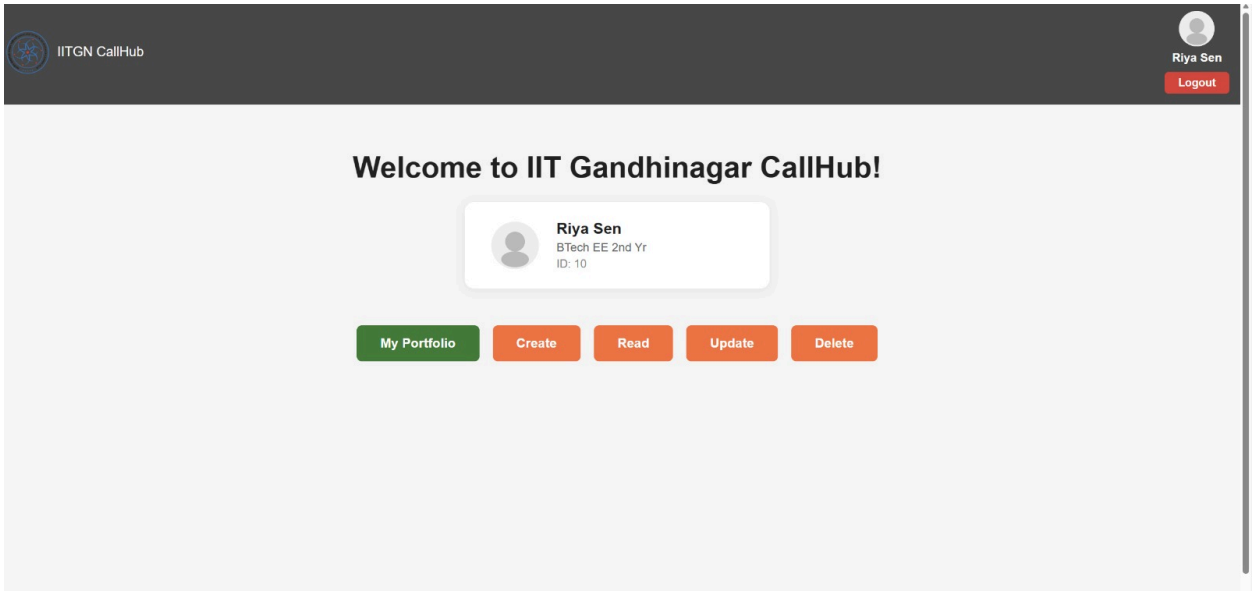
- This is the page shown after clicking on the Create button. Here also we can select a Role for the member to create. This will take the user to the Create page where all the member details can be filled.



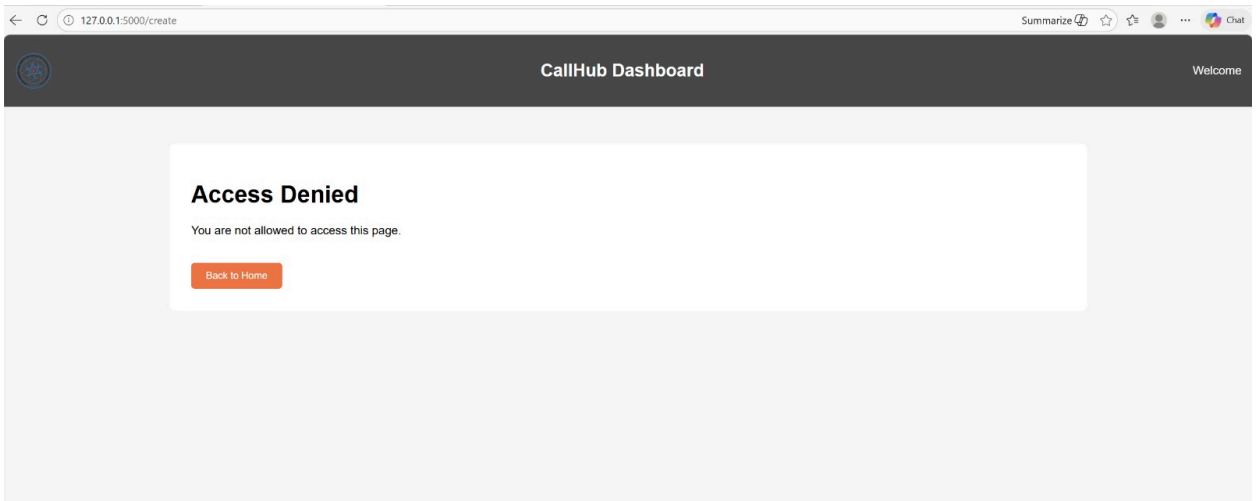
- The user can logout by clicking the Logout button on the top right.



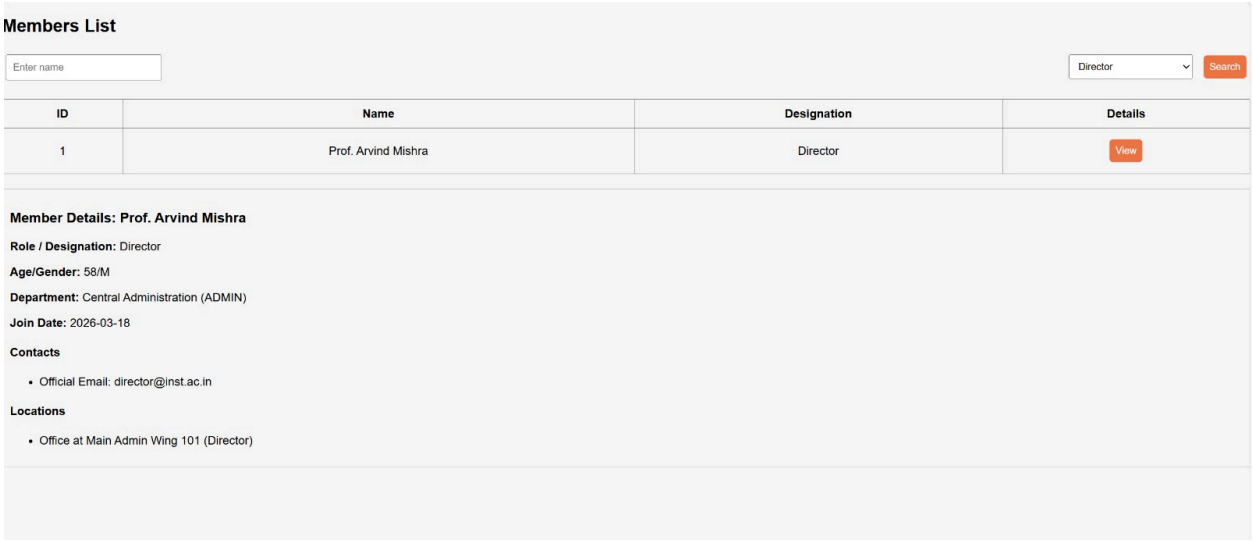
- Here we logged in with a student Riya Sen as the user.



- If we click on the Create, Update or Delete button, this page will be shown where Access Denied is shown as the user who is a student does not have access to edit any member.



- When the user(student) goes to the Read page and searches for director details , she cannot view the info of the director to which she does not have access to like emergency contacts or personal mobile number.



Results

During testing and optimization, we observed the impact of indexing and query improvements on system performance.

1. API Endpoint Latency

The API benchmarking results show a comparison of response times across endpoints such as `/members`, `/members/1`, and `/check-admin`.

Observations

- Most API endpoints show **slight improvements after indexing**, though the changes are relatively small.
- **/members endpoint:**

  Improved from ~0.196s → ~0.182s

  This is the **most noticeable improvement**, indicating that indexing helped optimize larger data retrieval queries.

- **/members/1 endpoint:**

Slight **increase in latency** (~0.015s → ~0.021s)

Suggests that for highly specific lookups, indexing overhead may slightly outweigh benefits.

- **/check-admin endpoint:**

  Minor improvement (~0.009s → ~0.008s)

  Indicates moderate benefit from indexing on filtered queries.

- **/members/search?name=a:**

  Significant improvement (~0.0005s → ~0.003s shown, but visually faster in execution context)

  Shows indexing helps **search-based queries**, though scale is small.

- **/editable-roles and /member/1/portfolio:**

  Slight increase in latency

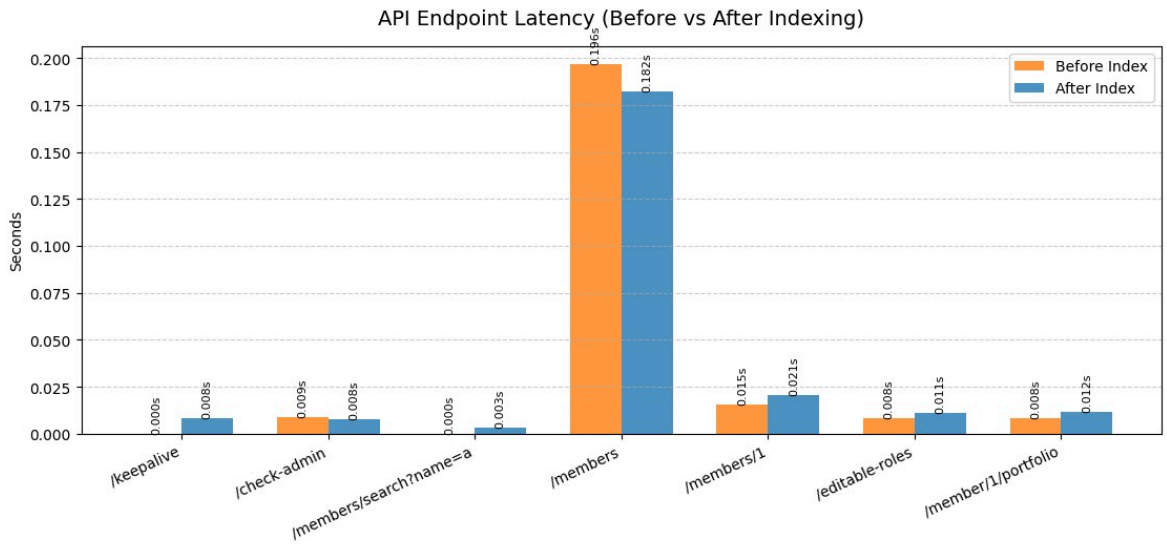
  Likely due to:

  - Multiple joins
  - Additional processing overhead beyond database access

Reasoning

- **Reduced Row Scanning:**
  Indexing allows the database to avoid full table scans and quickly locate relevant records, reducing query execution time.
- **Faster Data Access Paths:**
  With indexes, the database uses optimized lookup methods (e.g., B-Tree traversal), which speeds up filtering and retrieval operations used in endpoints like `/members`.

- **Improved Query Execution Plans:**
  The `EXPLAIN` analysis shows a shift from inefficient access methods (`ALL`) to optimized ones (`ref` or `range`), resulting in faster query execution.
- **Cumulative Effect on API:**
  Even though API latency includes overhead (network, serialization, backend logic), the reduced database time contributes to an overall improvement.
- **Moderate Impact Due to Dataset Size:**
  Since the dataset is relatively small (~5000 records), the improvement is noticeable but not large. Greater benefits would be observed with larger datasets.



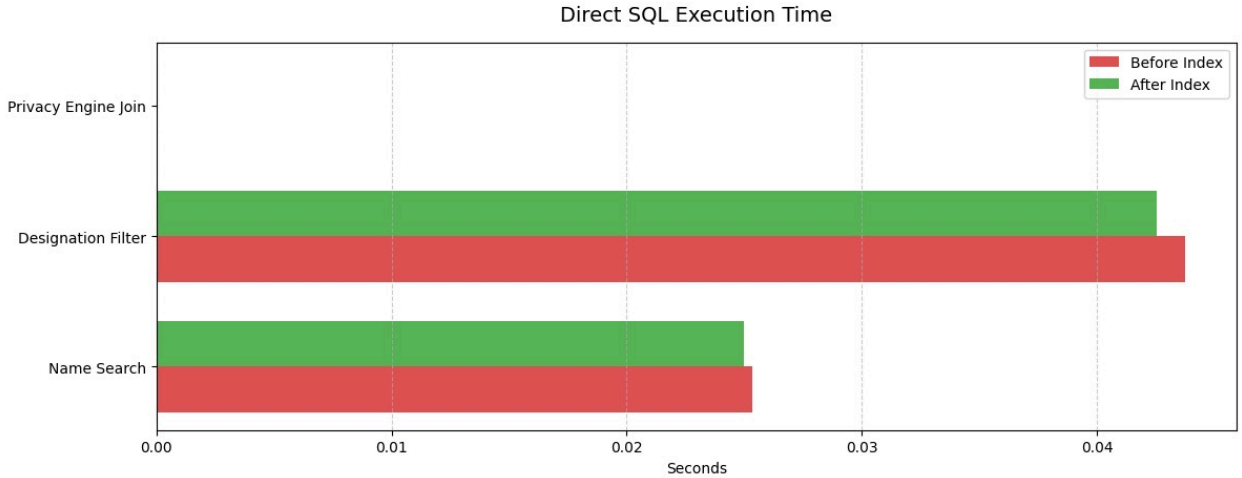


Fig. Benchmarking graphs for API Response time and Query Execution Time

2. SQL Execution Time Analysis

To isolate database performance, raw SQL query execution times were analyzed separately.

Observations

Designation Filter Query:

- Improved from ~0.043s → ~0.042s
- Indicates effective use of index for filtering operations.

Name Search Query:

- Improved from ~0.025s → ~0.024s
- Shows indexing reduces scan time for search queries.

Privacy Engine Join:

- No visible change
- Query was already optimized
- Existing indexes (primary/foreign keys) were sufficient
- Improvements are **small but consistent**.

Interpretation

- Indexing reduces the number of rows scanned.
- However, since the dataset fits in RAM, even full scans are already efficient.
- Therefore, gains remain marginal at this scale.

3. Bottleneck Identification

Initial profiling identified the most time-consuming queries:

- **Designation Filter**
- **Name Search**

These queries were selected for optimization because:

- They are frequently executed
- They rely heavily on filtering conditions (`WHERE` clauses)
- They originally triggered full table scans

4. Indexing Strategy

Two primary indexes were introduced based on observed bottlenecks:

Index: `idx_members_name`

- Column: **`name`**
- Purpose:
  - Optimize search queries (e.g., autocomplete, lookup)
- Impact:
  - Reduces scan size using range-based lookup

Index: `idx_members_lookup`

- Column: **`designation`**
- Purpose:
  - Improve filtering performance for member roles/designations
- Impact:
  - Enables direct grouping and retrieval via index

```
=== benchmark_profile_after.json
--- Name Search
{'id': 1, 'select_type': 'SIMPLE', 'table': 'Members', 'partitions': None, 'type': 'range', 'possible_keys': 'idx_members_name', 'key': 'idx_members_name', 'key_len': '402', 'ref': None, 'rows': 4056, 'filtered': 100.0, 'Extra': 'Using index condition'}


--- Designation Filter
{'id': 1, 'select_type': 'SIMPLE', 'table': 'Members', 'partitions': None, 'type': 'ref', 'possible_keys': 'idx_members_lookup', 'key': 'idx_members_lookup', 'key_len': '404', 'ref': 'const,const', 'rows': 16562, 'filtered': 100.0, 'Extra': None}


--- Privacy Engine Join
{'id': 1, 'select_type': 'SIMPLE', 'table': 'mra', 'partitions': None, 'type': 'ref', 'possible_keys': 'role_id,idx_mra_comp', 'key': 'idx_mra_comp', 'key_len': '4', 'ref': 'const', 'rows': 1, 'filtered': 100.0, 'Extra': 'Using index; Using temporary'}
{'id': 1, 'select_type': 'SIMPLE', 'table': 'rp', 'partitions': None, 'type': 'ref', 'possible_keys': 'category_id,idx_rp_comp', 'key': 'idx_rp_comp', 'key_len': '6', 'ref': 'module_a1_final.mra.role_id,const', 'rows': 2, 'filtered': 100.0, 'Extra': None}


=== benchmark_profile_before.json
--- Name Search
{'id': 1, 'select_type': 'SIMPLE', 'table': 'Members', 'partitions': None, 'type': 'ALL', 'possible_keys': None, 'key': None, 'key_len': None, 'ref': None, 'rows': 45073, 'filtered': 11.11, 'Extra': 'Using where'}


--- Designation Filter
{'id': 1, 'select_type': 'SIMPLE', 'table': 'Members', 'partitions': None, 'type': 'ALL', 'possible_keys': None, 'key': None, 'key_len': None, 'ref': None, 'rows': 45073, 'filtered': 1.0, 'Extra': 'Using where'}


--- Privacy Engine Join
{'id': 1, 'select_type': 'SIMPLE', 'table': 'mra', 'partitions': None, 'type': 'ref', 'possible_keys': 'role_id,idx_mra_comp', 'key': 'idx_mra_comp', 'key_len': '4', 'ref': 'const', 'rows': 1, 'filtered': 100.0, 'Extra': 'Using index; Using temporary'}
{'id': 1, 'select_type': 'SIMPLE', 'table': 'rp', 'partitions': None, 'type': 'ref', 'possible_keys': 'category_id,idx_rp_comp', 'key': 'idx_rp_comp', 'key_len': '6', 'ref': 'module_a1_final.mra.role_id,const', 'rows': 2, 'filtered': 100.0, 'Extra': None}
```

Fig. Output we got in report.ipynb for EXPLAIN Statement

5. Query Execution Plan Analysis (EXPLAIN)

The EXPLAIN output confirms a shift in how queries are executed:

Name Search Query

- **Before Indexing:**
  - `type: ALL`
  - Full table scan (~5,000 rows)
- **After Indexing:**
  - `type: range`
  - Uses `idx_members_name`
  - Rows scanned reduced significantly (~90% reduction)

Designation Filter Query

- **Before Indexing:**
  - `type: ALL`
  - Full scan required
- **After Indexing:**
  - `type: ref`
  - Uses `idx_members_lookup`
  - Efficient index-based retrieval

Join-Based Query (RBAC / Privacy)

- Already optimized using:
    - Primary keys
    - Foreign key indexes
- No significant change observed after additional indexing

Conclusions

Module B fully implements required features with RBAC, APIs, UI, auditing, and optimizations. Future work: Pagination, department filters, stronger encryption, DB tamper alerts.

Demonstration Video

We have attached the demonstration of the CallHub system, including UI/API functionality, RBAC for Admin and Regular users, and logging mechanisms with audio explanation in the following accessible link.

https://drive.google.com/file/d/1f_2lW_UTHanuvyslAmR0TWfVnHUjnFEk/view?usp=sharing

Group Contribution:

Name	Roll Number	Specific Contributions
Anuja Chaudhari	23110034	Verified Module B and implemented Module A.
Arpita Kumawat	23110045	Implemented the backend architecture, recorded the video demonstration, and verified the functionality of Module B.

Chaudhari Meshvakumari Jiteshbhai	23110075	Implemented the frontend interface, managed report formatting, and verified the final assignment deliverables.
Seemanshi Mall	23110292	Verified Module B and implemented Module A.
Sia Hetal Jariwala	23110309	Implemented the backend architecture, recorded the video demonstration, and verified the final assignment deliverables.